

编译原理-中间代码生成

语法制导翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 5 月 13 日



福州大学
FUZHOU UNIVERSITY



- ① 知识点回顾
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储



- ① 知识点回顾
编译程序的阶段划分
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储



- ① 知识点回顾
编译程序的阶段划分
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储

C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);



C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编” 即整理, “译” 即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;

C 语言编译器的典型阶段

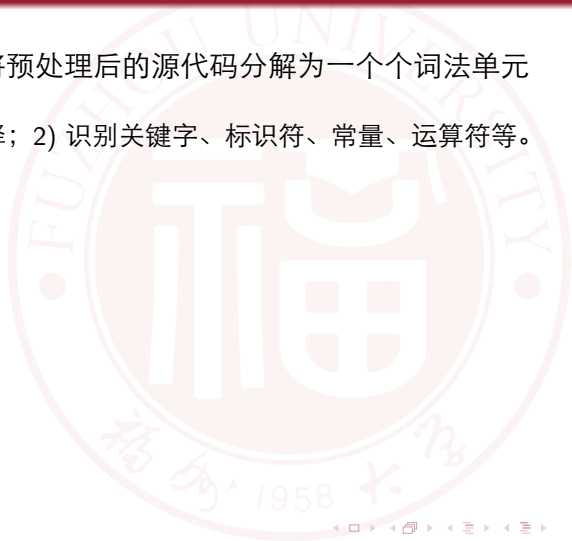
- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编” 即整理, “译” 即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;

C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理, “译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;
- ④ 链接阶段 (Linking): 将一个或多个二进制目标文件进行关联, 处理外部函数跳转地址, 全局变量初始化等以生成可执行文件。

编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
- 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。



编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。

编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- ③ 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

编译阶段（续）

- ④ 中间代码生成（Intermediate Code Generation）：将抽象语法树转换为中间代码（Intermediate Representation, IR）。中间代码是一种与硬件无关的低级表示形式（如三地址码），以便于后续优化和目标代码生成。

编译阶段（续）

- ④ 中间代码生成（Intermediate Code Generation）：将抽象语法树转换为中间代码（Intermediate Representation, IR）。中间代码是一种与硬件无关的低级表示形式（如三地址码），以便于后续优化和目标代码生成。
- ⑤ 优化阶段（Optimization）：对中间代码进行优化，提高程序的运行效率。
- 过程：常量折叠、公共子表达式消除、循环优化等。
 - 分类：前端优化（与语言相关）和后端优化（与硬件相关）。
 - 输出：优化后的中间代码。

编译阶段（续）

- ④ 中间代码生成（Intermediate Code Generation）：将抽象语法树转换为中间代码（Intermediate Representation, IR）。中间代码是一种与硬件无关的低级表示形式（如三地址码），以便于后续优化和目标代码生成。
- ⑤ 优化阶段（Optimization）：对中间代码进行优化，提高程序的运行效率。
 - 过程：常量折叠、公共子表达式消除、循环优化等。
 - 分类：前端优化（与语言相关）和后端优化（与硬件相关）。
 - 输出：优化后的中间代码。
- ⑥ 目标代码生成（Code Generation）：将优化后的中间代码转换为目标代码（机器语言）。
 - 过程：处理寄存器分配、指令选择等。
 - 输出：目标代码（如汇编、Java 字节码、Python 字节码等）。

④ 三地址码的表示与存储



- ① 知识点回顾
- ② 中间代码生成阶段
定义与作用
如何分类?
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储

- 定义：中间代码是源程序在编译过程中的一种**中间表示（Intermediate Representation, IR）形式**。



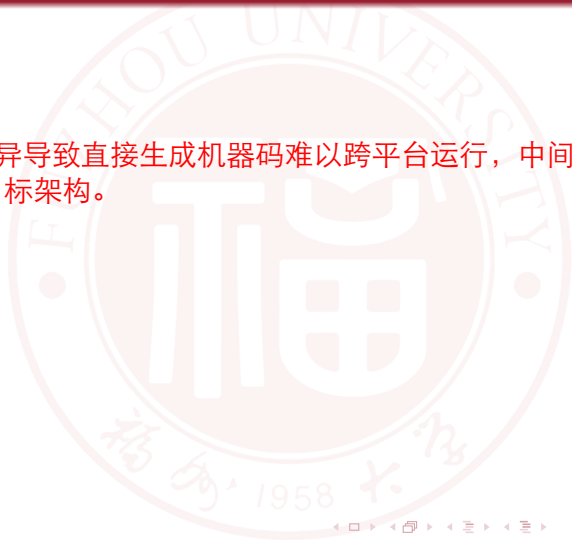
- 定义：中间代码是源程序在编译过程中的一种**中间表示（Intermediate Representation, IR）形式**。
- 特征：中间代码介于**高级语言**与**机器码**之间，具有**结构简单**、**机器无关性强**的特征。



- 定义：中间代码是源程序在编译过程中的一种**中间表示（Intermediate Representation, IR）形式**。
- 特征：中间代码介于**高级语言**与**机器码**之间，具有**结构简单**、**机器无关性强**的特征。
- 目的：引入中间代码的核心目的是增强编译器的**可移植性**，并为**代码优化**提供结构化支持。

中间代码的必要性

- 可移植性：不同 CPU 指令集差异导致直接生成机器码难以跨平台运行，中间代码作为统一接口可适配多种目标架构。



中间代码的必要性

- 可移植性：不同 CPU 指令集差异导致直接生成机器码难以跨平台运行，中间代码作为统一接口可适配多种目标架构。
- 代码优化：中间代码支持与机器无关的优化操作（如常量折叠、公共子表达式消除），提升生成目标代码的效率。

中间代码的必要性

- 可移植性：不同 CPU 指令集差异导致直接生成机器码难以跨平台运行，中间代码作为统一接口可适配多种目标架构。
- 代码优化：中间代码支持与机器无关的优化操作（如常量折叠、公共子表达式消除），提升生成目标代码的效率。
- 开发维护：分离编译器的前端（语法 / 语义分析）与后端（目标代码生成），降低模块耦合度，简化开发流程。

有无 IR 的示例

Pascal ————— MIPS

C

ARM

C++

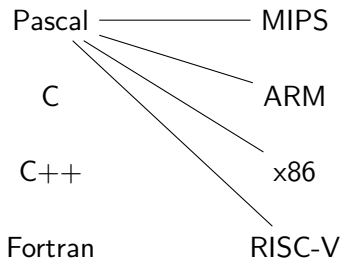
x86

Fortran

RISC-V

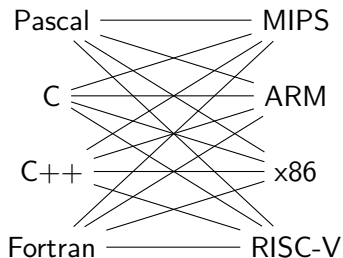


有无 IR 的示例



A diagram showing connections between programming languages and architectures. On the left, the languages are listed: Pascal, C, C++, and Fortran. On the right, the architectures are listed: MIPS, ARM, x86, and RISC-V. Lines connect the languages to the architectures as follows: Pascal connects to MIPS; C connects to MIPS, ARM, and x86; C++ connects to ARM, x86, and RISC-V; Fortran connects to RISC-V.

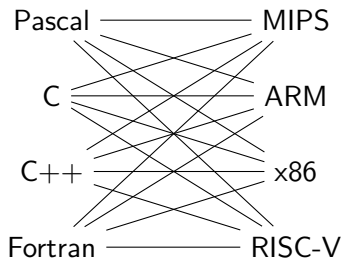
有无 IR 的示例



共需开发**16**个编译器。



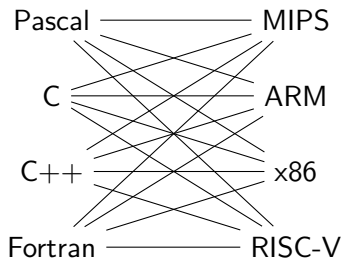
有无 IR 的示例



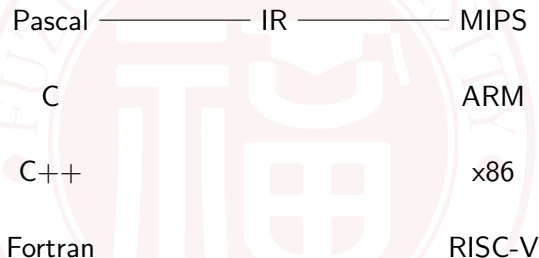
共需开发**16**个编译器。



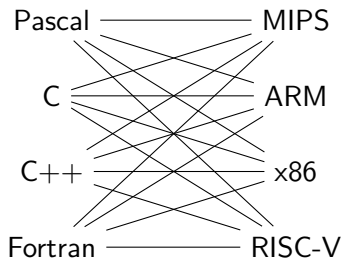
有无 IR 的示例



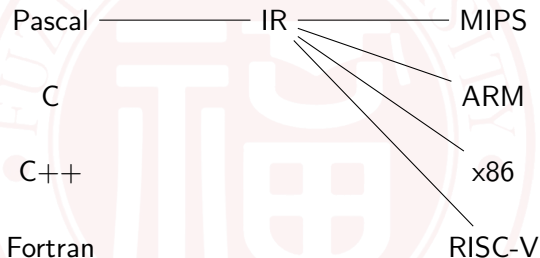
共需开发**16**个编译器。



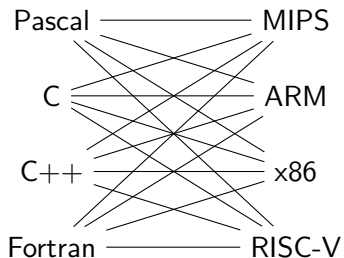
有无 IR 的示例



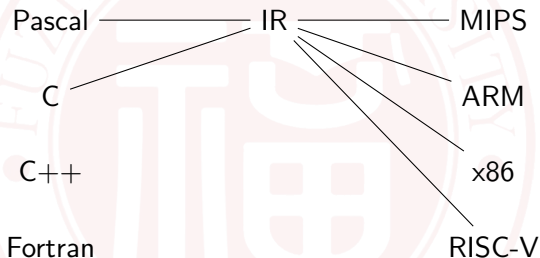
共需开发**16**个编译器。



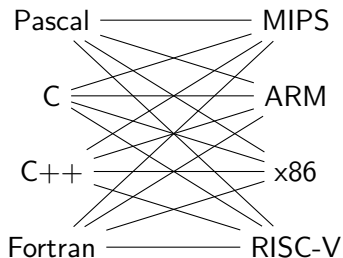
有无 IR 的示例



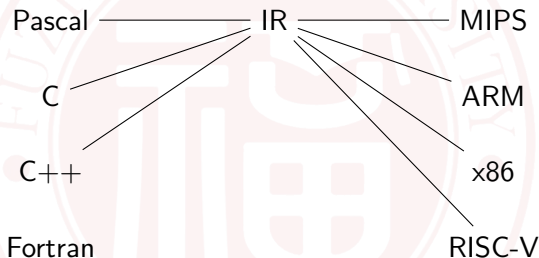
共需开发**16**个编译器。



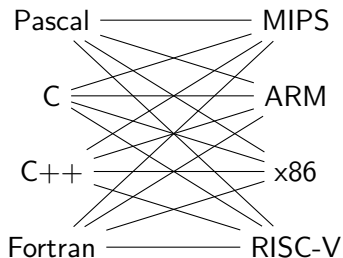
有无 IR 的示例



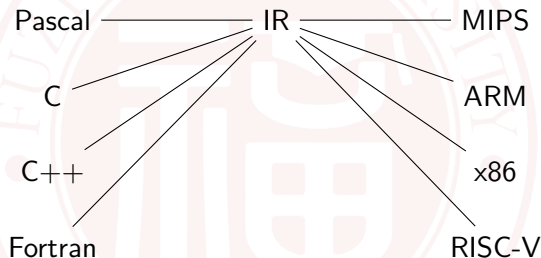
共需开发**16**个编译器。



有无 IR 的示例



共需开发**16**个编译器。



仅需开发**4**个前端和**4**个后端。



- ① 知识点回顾
- ② 中间代码生成阶段
定义与作用
如何分类?
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储

中间表示的功能与层次

按照 IR 的层次进行划分有：

- 高层次 IR (HIR)：能够准确表达源语言的语义，用于发现语法错误、分析符号之间的依赖关系（以便进行跳转、判断方法的重载等）、根据需要自动生成或修改一些代码（提供重构能力）。
- 中间层次 IR (MIR)：大量的优化算法是可以通用的，没有必要依赖源语言的语法和语义，也没有必要依赖具体的 CPU 架构。这些优化包括部分算术优化、常量和变量传播、死代码删除等，实现这些类分析和优化功能的 IR 可以叫做 Middle IR, 简称 MIR.
- 低层次 IR (LIR)：它的指令通常可以与机器指令一一对应，比较容易翻译成机器指令 (或汇编代码)。因为 LIR 体现了 CPU 架构的底层特征，因此可以做一些于具体 CPU 架构相关的优化。

H/M/LIR 示例

源代码

```
float a[10][20];  
a[i][j+2];
```

HIR

```
t1=a[i,j+2];
```

MIR

```
t1=j+2;  
t2=i*20;  
t3=t1+t2;  
t4=4*t3;  
t5=addr a;  
t6=t5+t4;  
t7=*t6;
```

LIR

```
r1=[fp-4];  
r2=[r1+2];  
r3=[fp-8];  
r4=r3*20;  
r5=r4+t2;  
r6=4*r5;  
r7=fp-216;  
f1=[r7+r6];
```



① 知识点回顾

② 中间代码生成阶段

③ 中间代码的主要形式

后缀式

树/图结构

三地址码

④ 三地址码的表示与存储

- 逆波兰式（后缀式）
- 图结构表示
- 三地址码



- ① 知识点回顾
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
 后缀式
 树/图结构
 三地址码
- ④ 三地址码的表示与存储

逆波兰式（后缀式）

- 逆波兰表达式（Reverse Polish Notation, RPN）是一种数学表达式的书写形式。

逆波兰式（后缀式）

- 逆波兰表达式（Reverse Polish Notation, RPN）是一种数学表达式的书写形式。
- 它由波兰数学家扬·武卡谢维奇于 1920 年提出。

逆波兰式（后缀式）

- 逆波兰表达式（Reverse Polish Notation, RPN）是一种数学表达式的书写形式。
- 它由波兰数学家扬·武卡谢维奇于 1920 年提出。
- 核心特征为运算符位于所有相关操作数之后，例如表达式“ $3+4$ ”的逆波兰式为“ $3\ 4\ +$ ”。

otation, RPN) 是一种数学表达式的书写形式，于 1920 年提出。

操作数之后，例如表达式 “3+4” 的逆波兰式

且运算符的优先级由操作符位置隐式确定，使

化。

- otation, RPN) 是一种数学表达式的书写形式，于 1920 年提出。
- 操作数之后，例如表达式 “3+4” 的逆波兰式
- 且运算符的优先级由操作符位置隐式确定，使
- 化。

后缀式的定义

- 一个表达式 E 的后缀形式可以如下定义：



后缀式的定义

- 一个表达式 E 的后缀形式可以如下定义：
 - 若 E 是一个变量或常量，则 E 的后缀式是 E 自身。



后缀式的定义

- 一个表达式 E 的后缀形式可以如下定义：
 - 若 E 是一个变量或常量，则 E 的后缀式是 E 自身。
 - 若 E 是形如 $E_1 \text{ op } E_2$ 的表达式，其中 op 是任何二元操作符，则 E 的后缀式为 $E_1 E_2 \text{ op}$ ，其中 E_1 和 E_2 分别为 E_1 和 E_2 的后缀式。

后缀式的定义

- 一个表达式 E 的后缀形式可以如下定义：
 - 若 E 是一个变量或常量，则 E 的后缀式是 E 自身。
 - 若 E 是形如 $E_1 \text{ op } E_2$ 的表达式，其中 op 是任何二元操作符，则 E 的后缀式为 $E_1 E_2 \text{ op}$ ，其中 E_1 和 E_2 分别为 E_1 和 E_2 的后缀式。
 - 若 E 是形如 (E_1) 的表达式，则 E_1 的后缀式就是 E 的后缀式。

后缀式的生成过程

将中缀表达式转换为逆波兰式的算法遵循以下步骤：

- 操作数处理：遇到数字或变量直接输出至结果序列。
- 运算符与栈操作：
 - 左括号'('：直接压入操作符栈。
 - 右括号')'：弹出栈顶元素至结果序列，直至遇到左括号（左括号弹出但不输出）。
 - 优先级比较：若当前运算符优先级低于或等于栈顶运算符，则弹出栈顶元素至结果序列，直至栈顶运算符优先级更低或遇到左括号，再将当前运算符入栈。
- 表达式扫描完成：将栈中剩余运算符依次弹出至结果序列。

后缀式示例

输入： 2 + 3 * 4

输出：

符号栈：



后缀式示例

输入： + 3 * 4

输出： 2

符号栈：



后缀式示例

输入:

3 * 4

输出: 2

符号栈: +



后缀式示例

输入：

输出： 2

符号栈：+

3

*

4



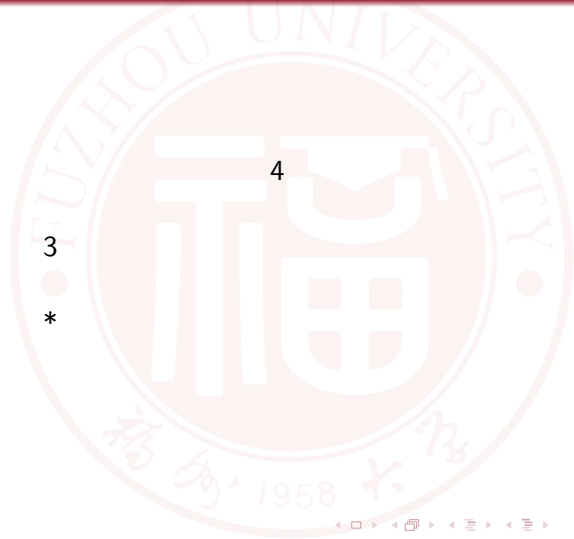
后缀式示例

输入:

输出: 2

符号栈: +

4
3
*



后缀式示例

输入:

输出: 2 3 4

符号栈: + *



后缀式示例

输入:

输出: 2 3 4 *

符号栈: +

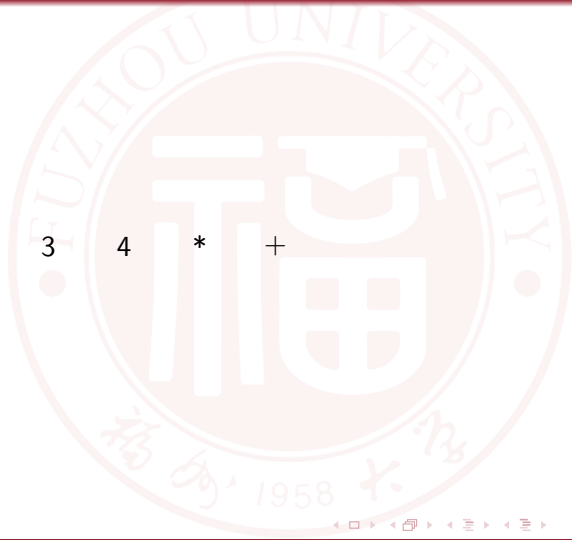


后缀式示例

输入:

输出: 2 3 4 * +

符号栈:

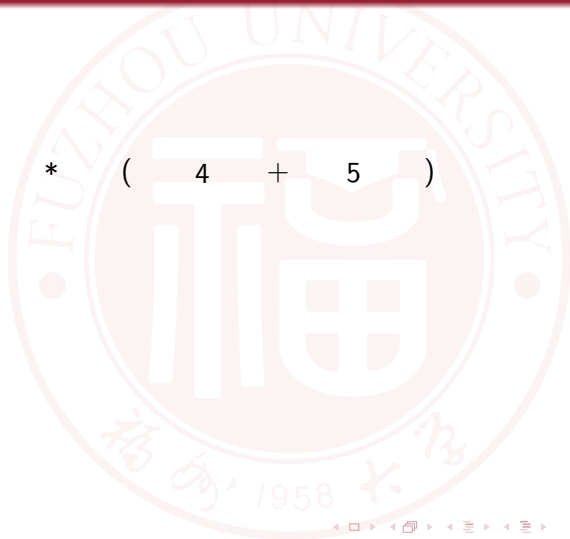


后缀式示例

输入： 2 + 3 * (4 + 5)

输出：

符号栈：

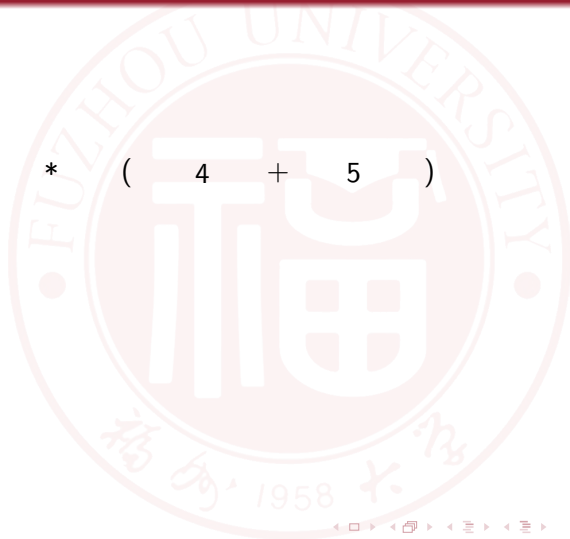


后缀式示例

输入: + 3 * (4 + 5)

输出: 2

符号栈:



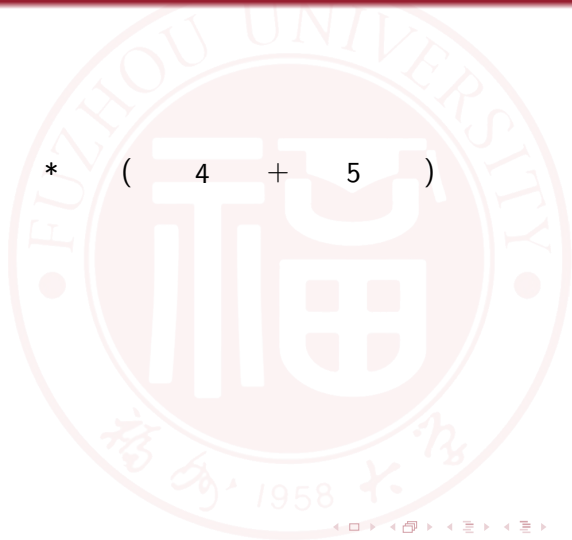
后缀式示例

输入:

3 * (4 + 5)

输出: 2

符号栈: +



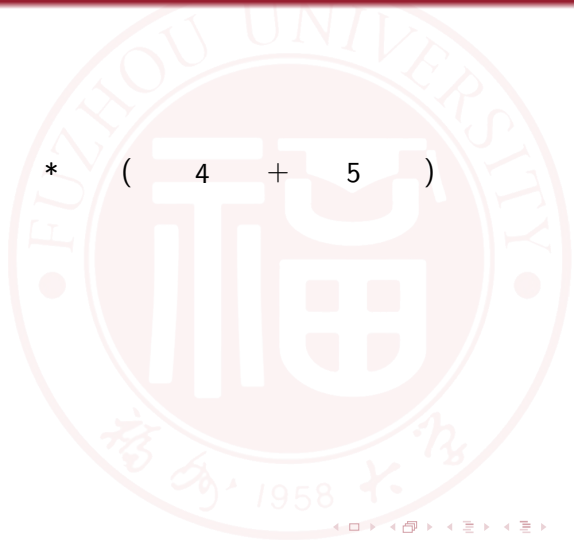
后缀式示例

输入:

* (4 + 5)

输出: 2 3

符号栈: +



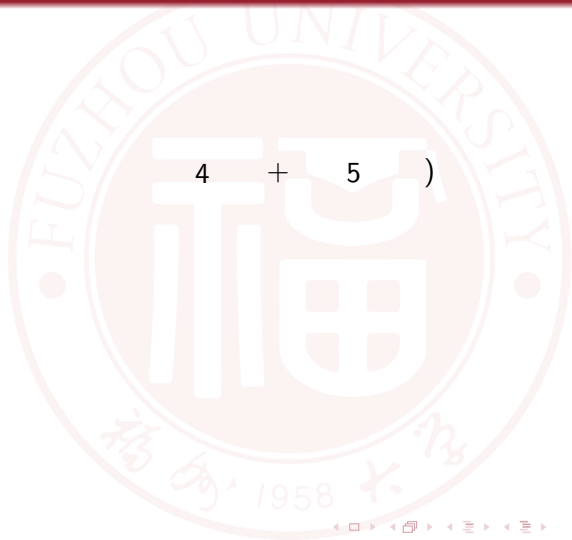
后缀式示例

输入:

4 + 5)

输出: 2 3

符号栈: + *



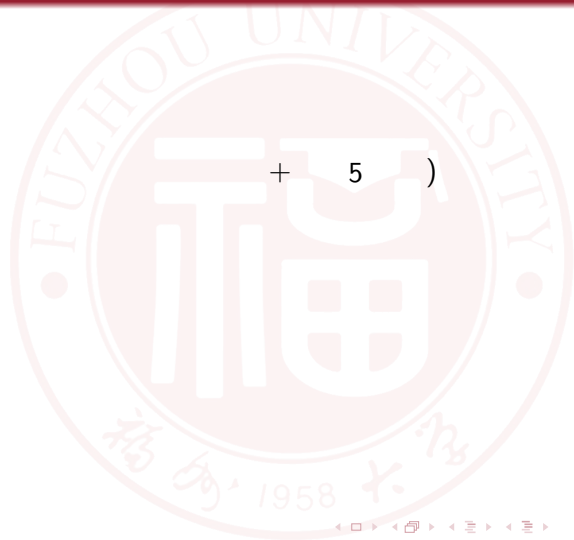
后缀式示例

输入:

+ 5)

输出: 2 3 4

符号栈: + *



后缀式示例

输入:

输出: 2 3 4

符号栈: + * +



后缀式示例

输入:

)

输出: 2 3 4 5

符号栈: + * +



后缀式示例

输入:

输出: 2 3 4 5

符号栈: + * +

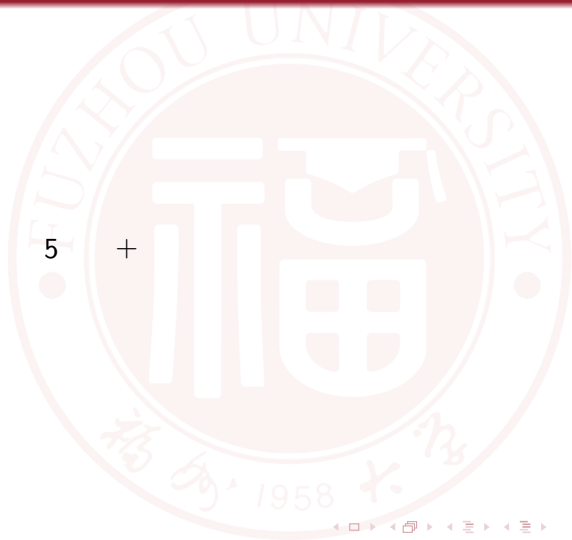


后缀式示例

输入:

输出: 2 3 4 5 +

符号栈: + *



后缀式示例

输入:

输出: 2 3 4 5 + *

符号栈: +

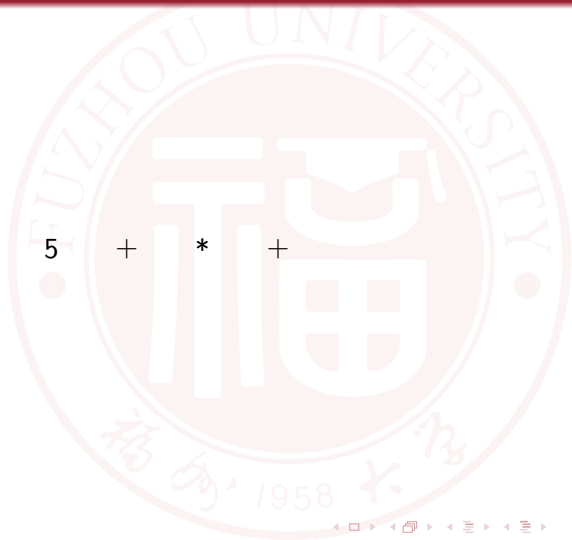


后缀式示例

输入:

输出: 2 3 4 5 + * +

符号栈:



翻译成后缀式的属性文法

产生式	语义规则
$E \rightarrow E_1 \text{ op } E_2$	$E.code = E_1.code E_2.code \text{op}$

其中， $E.code$ 表示 E 后缀形式， op 表示任意二元操作符，“ $||$ ”表示后缀形式的连接。

对应的 SDT 方案为

语义规则

$$E.code = E_1.code || E_2.code || op$$

表示任意二元操作符，“||”表示后缀形式的连接

$$code = E_1.code || E_2.code || op \}$$

产生式	语义规则
$E \rightarrow E_1 \text{ op } E_2$	$E.code = E_1.code E_2.code \text{op}$

其中, $E.code$ 表示 E 后缀形式, op 表示任意二元操作符, “||”表示后缀形式的连接。

对应的 SDT 方案为

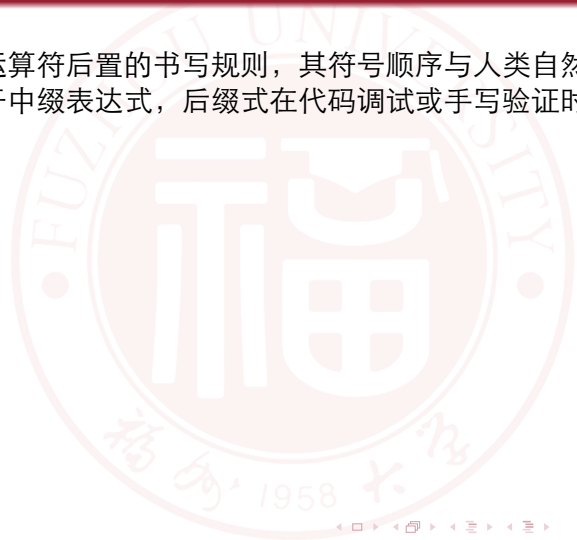
$$E \rightarrow E_1 \text{ op } E_2 \{ E.code = E_1.code || E_2.code || \text{op} \}$$

后缀式的执行过程

- 后缀式的计算依赖栈数据结构实现，具体流程如下：
 - 操作数入栈：从左至右扫描表达式，遇到操作数直接压入栈。
 - 运算符处理：遇到运算符时，弹出栈顶两个操作数执行运算，并将结果压回栈中。
 - 终止条件：表达式扫描完毕后，栈顶元素即为最终计算结果。

后缀式的局限

- **人工可读性不足**：后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。



后缀式的局限

- **人工可读性不足**：后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。
- **转换过程的计算开销**：对于大规模表达式或高频转换场景（如动态生成代码的场景），可能引发额外的性能损耗。在资源受限的嵌入式系统中仍需权衡效率。

后缀式的局限

- **人工可读性不足**：后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。
- **转换过程的计算开销**：对于大规模表达式或高频转换场景（如动态生成代码的场景），可能引发额外的性能损耗。在资源受限的嵌入式系统中仍需权衡效率。
- **特定场景的结构局限性**：在单线程顺序计算中表现高效，但对并行计算或硬件级优化的支持较弱。现代处理器常利用指令级并行技术（ILP）对独立运算单元加速，而逆波兰式的线性栈结构难以显式表达操作间的独立性，可能限制硬件优化潜力。对带有副作用（如赋值、函数调用）的表达式处理时，需引入额外标记或中间状态，增加实现复杂度。

后缀式的局限

- **人工可读性不足**：后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。
- **转换过程的计算开销**：对于大规模表达式或高频转换场景（如动态生成代码的场景），可能引发额外的性能损耗。在资源受限的嵌入式系统中仍需权衡效率。
- **特定场景的结构局限性**：在单线程顺序计算中表现高效，但对并行计算或硬件级优化的支持较弱。现代处理器常利用指令级并行技术（ILP）对独立运算单元加速，而逆波兰式的线性栈结构难以显式表达操作间的独立性，可能限制硬件优化潜力。对带有副作用（如赋值、函数调用）的表达式处理时，需引入额外标记或中间状态，增加实现复杂度。
- **语法扩展的适配难度**：需同步扩展优先级规则和栈处理逻辑，这对动态类型语言或元编程场景的灵活性构成挑战。



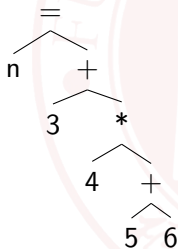
- ① 知识点回顾
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
 后缀式
 树/图结构
 三地址码
- ④ 三地址码的表示与存储

图结构 IR

图结构 IR 按照实际的图的类型有可以分为**树状中间表示 (Tree-based IR)** 和**图结构中间表示 (Graph-based IR)**。

树状中间表示 (Tree-based IR)

- 树状中间表示 (Tree-based IR) 是虎书¹所使用的一种中间表示方法。
- 树状中间表示以树形数据结构为基础，通常由抽象语法树 (AST) 演化而来。每个节点代表一个操作 (如算术运算、函数调用等)，子节点表示操作数或子表达式。



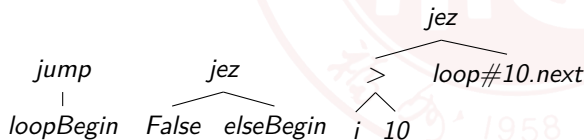
¹Modern Compiler Implementation in Java/C++/ML, Second Edition, Andrew W. Appel & Jens Palsberg

问题

- 一个程序几棵树？
- 复杂结构（如分支、循环）怎么表述？

解

- 一个程序的树状 *IR* 通常由一个森林构成（包括多个独立的树）。
- 每个赋值语句、无结果函数调用、循环、分支等作为一棵单独的 *IR* 树。根结点之间通过 *.next* 属性串联。
- 把指令位置/标签（*Label*）也是视为一种类型的数据。



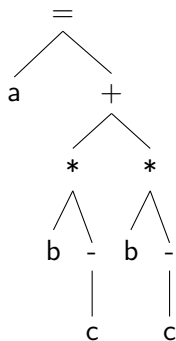
树形 IR 的优劣势

- 层次化结构：严格遵循程序的语法结构，便于语法分析和语义检查。
- 显式操作关系：通过父子节点关系直接体现操作符与操作数的绑定顺序。
- 局部性优化：适合局部表达式优化（如常量折叠），但对跨作用域或复杂控制流的支持较弱。

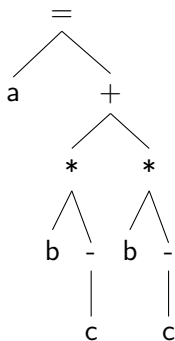
转 AST 的属性文法

$E \rightarrow E1 + T \{E.node = \text{new node}(\text{"+"}, E1.node, T.node)\}$
 $E \rightarrow E1 - T \{E.node = \text{new node}(\text{"-"}, E1.node, T.node)\}$
 $E \rightarrow T \{E.node = T.node\}$
 $T \rightarrow (E) \{T.node = E.node\}$
 $T \rightarrow id \{T.node = \text{new leaf}(id, id.lexval)\}$
 $T \rightarrow num \{T.node = \text{new leaf}(num, num.val)\}$

树结构的局限 $(a=b*(-c)+b*(-c))$ 的 IR



树结构的局限 $(a=b*(-c)+b*(-c))$ 的 IR



图结构 IR

图结构 IR 通过节点（表示操作或数据）和边（表示控制流或数据依赖）构建程序模型。典型形式包括：

- 控制流图（CFG）：基本块作为节点，边表示程序执行路径的跳转。
- 数据流图（DFG）：节点为操作，边表示数据依赖关系。
- 静态单赋值（SSA）：变量仅赋值一次，通过 Phi 函数处理分支合并。

构建 DAG 的值编码方法

- 语法树或 DAG 图的结点通常存放在一个记录数组中
 - 数组的每一行是一个记录，表示一个结点；
 - 数组下标为对应结点的值编码；
 - 叶结点包含记号字段和一个附加字段，存放词法值（标识符时为指向符号表相应项目的指针，数字时是常量）；
 - 内部结点包含运算符字段和若干附加字段，指向其儿子结点；



1	id	0x123	
2	num	10	
3	+	1	2
4	=	1	3
	⋮		

应用场景及优缺点

图结构 IR 的主要应用场景：

- 编译器优化：常量传播、死代码消除、循环优化等需全局分析的场景。
- 自动微分与机器学习：计算图支持梯度计算和分布式训练，例如在深度学习框架中实现反向传播。
- 并行化处理：通过数据依赖分析识别可并行执行的任务。

图结构 IR 的优缺点：

- 优点：高效支持复杂优化和跨过程分析；适合现代硬件加速和并行计算。
- 缺点：构建和维护成本较高；内存消耗较大，调试复杂度提升。

树 IR 和图结构 IR 总结

- 现代编译器常采用混合策略：前端使用树状 IR 进行语法和语义分析，后端转换为图结构 IR 实现深度优化。例如，LLVM 在生成低级中间表示（LLVM IR）前，可能先构建控制流图以支持循环优化和指令调度。
- 树状 IR 与图结构 IR 分别服务于编译过程的不同需求。树状 IR 以简洁性和直观性见长，适合前端处理；图结构 IR 以强大的分析和优化能力为核心，是后端优化的基石。理解两者的设计哲学和应用边界，对编译器开发及中间表示的选择至关重要。



- ① 知识点回顾
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
 - 后缀式
 - 树/图结构
 - 三地址码
- ④ 三地址码的表示与存储

39 / 53

三地址码 (Triple Address Code, TAC) 是一种中间表示形式, 每条指令最多包含三个操作数, 其基本结构可分解为四元组形式:

(运算符, 操作数 1, 操作数 2, 结果)

表达式 $a = b + c * d$ 可转换为:

$$t1 = c * d$$
$$a = b + t1$$

三地址码的设计目标：通过引入临时变量分解复合表达式，简化语法树或图结构的复杂性，为后续优化和机器无关代码生成提供线性化、标准化的中间表示。

三地址码

三地址码（Triple Address Code, TAC）是一种中间表示形式，每条指令最多包含三个操作数，其基本结构可分解为四元组形式：

（运算符，操作数 1，操作数 2，结果）

例

表达式 $a = b + c * d$ 可转换为：

$t1 = c * d$

$a = b + t1$

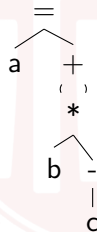
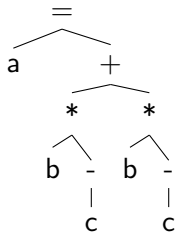
三地址码的设计目标：通过引入临时变量分解复合表达式，简化语法树或图结构的复杂性，为后续优化和机器无关代码生成提供线性化、标准化的中间表示。

“三”是一个**泛指**，具体由**四**元式、**三**元式、间接**三**元式等多种形式。

Pay Attention!!!

三地址码既可以是树形 IR（如 AST）的线性表示，也可以是 DAG 的线性表示。

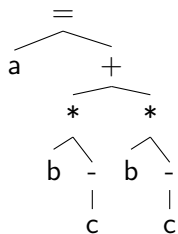
例



Pay Attention!!!

三地址码既可以是树形 IR（如 AST）的线性表示，也可以是 DAG 的线性表示。

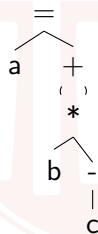
例



AST 的三地址码：

```

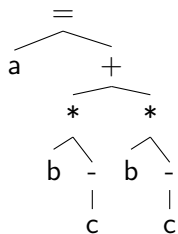
t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
    
```



Pay Attention!!!

三地址码既可以是树形 IR（如 AST）的线性表示，也可以是 DAG 的线性表示。

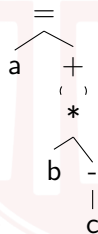
例



AST 的三地址码:

```

t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
    
```



DAG 的三地址码:

```

t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
    
```

常见三地址码

- 赋值指令：



常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;



常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = \text{op } y$: op 是单目运算符;



常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = \text{op } y$: op 是单目运算符;
 - $x = y$: 复制指令;



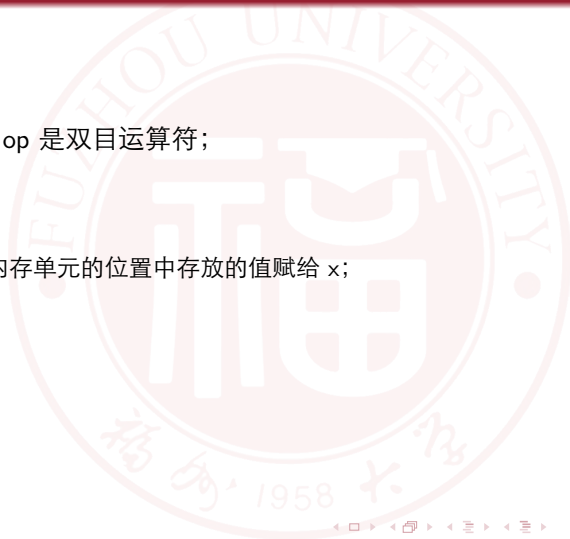
常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = \text{op } y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:



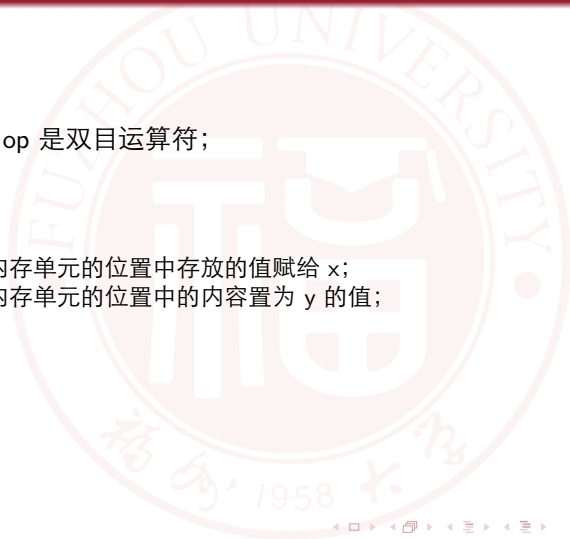
常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = \text{op } y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;



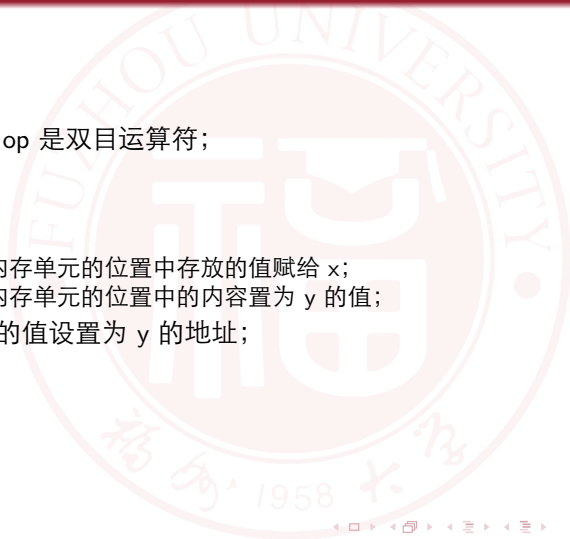
常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = op \ y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;
 - $x[i] = y$: 将距离 x 处 i 个内存单元的位置中的内容置为 y 的值;



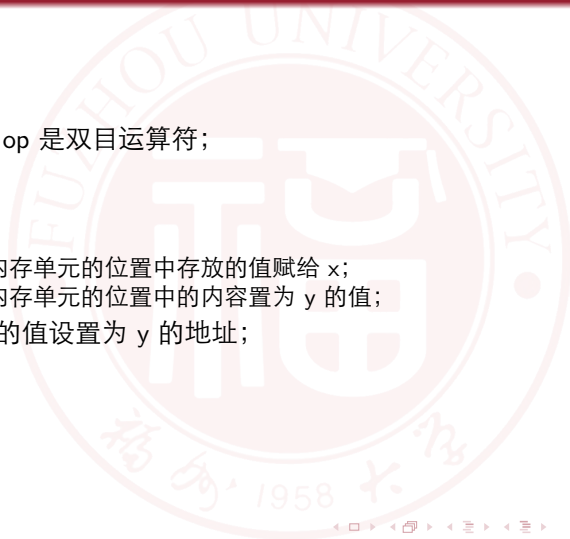
常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = \text{op } y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;
 - $x[i] = y$: 将距离 x 处 i 个内存单元的位置中的内容置为 y 的值;
- 地址赋值指令 $x = \&y$: 将 x 的值设置为 y 的地址;



常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = op \ y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;
 - $x[i] = y$: 将距离 x 处 i 个内存单元的位置中的内容置为 y 的值;
 - 地址赋值指令 $x = \&y$: 将 x 的值设置为 y 的地址;
 - 指针赋值指令:



常见三地址码

● 赋值指令：

- $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
- $x = op \ y$: op 是单目运算符;
- $x = y$: 复制指令;
- 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;
 - $x[i] = y$: 将距离 x 处 i 个内存单元的位置中的内容置为 y 的值;
- 地址赋值指令 $x = \&y$: 将 x 的值设置为 y 的地址;
- 指针赋值指令:
 - $x = *y$: 将 x 的值置为地址 y 的值;

常见三地址码

- 赋值指令：
 - $x = y \text{ op } z$: x, y, z 是地址, op 是双目运算符;
 - $x = op \ y$: op 是单目运算符;
 - $x = y$: 复制指令;
 - 带下标的复制指令:
 - $x = y[i]$: 将距离 y 处 i 个内存单元的位置中存放的值赋给 x ;
 - $x[i] = y$: 将距离 x 处 i 个内存单元的位置中的内容置为 y 的值;
 - 地址赋值指令 $x = \&y$: 将 x 的值设置为 y 的地址;
 - 指针赋值指令:
 - $x = *y$: 将 x 的值置为地址 y 的值;
 - $*x = y$: 把 y 的值赋到地址 x ;

常见三地址码

- 转移指令：



- 转移指令：
 - 无条件转移指令 `goto L`: 下一步执行带有标号 `L` 的三地址指令;

常见三地址码

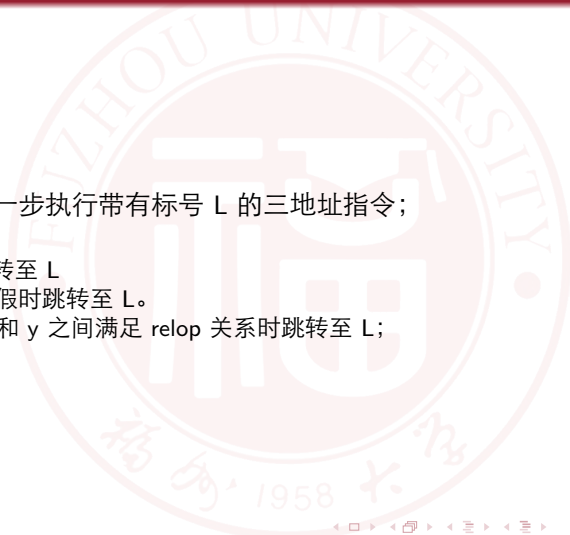
- 转移指令：
 - 无条件转移指令 `goto L`: 下一步执行带有标号 `L` 的三地址指令；
 - 条件转移指令：
 - `if x goto L`: `x` 为真时跳转至 `L`

常见三地址码

- 转移指令：
 - 无条件转移指令 goto L: 下一步执行带有标号 L 的三地址指令；
 - 条件转移指令：
 - if x goto L: x 为真时跳转至 L
 - if False x goto L: x 为假时跳转至 L。

常见三地址码

- 转移指令：
 - 无条件转移指令 goto L: 下一步执行带有标号 L 的三地址指令；
 - 条件转移指令：
 - if x goto L: x 为真时跳转至 L
 - if False x goto L: x 为假时跳转至 L。
 - if x relop y goto L: x 和 y 之间满足 relop 关系时跳转至 L；



常见三地址码

- 过程调用及返回：



常见三地址码

- 过程调用及返回：
 - 参数传递 param x: 传递参数 x;



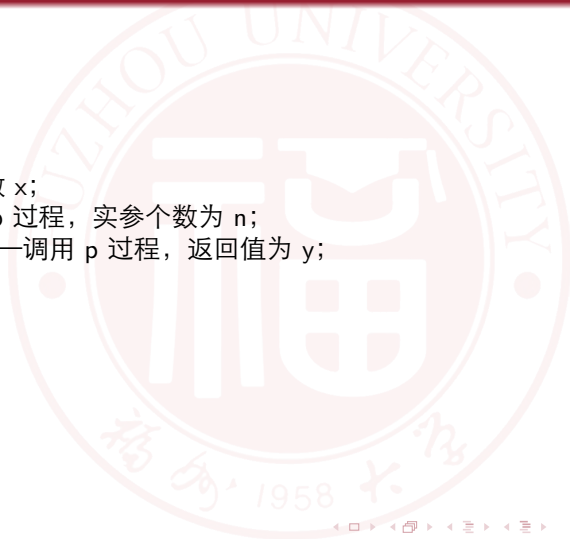
常见三地址码

- 过程调用及返回：
 - 参数传递 `param x`: 传递参数 `x`;
 - 过程调用 `call p, n`: 调用 `p` 过程, 实参个数为 `n`;



常见三地址码

- 过程调用及返回：
 - 参数传递 `param x`: 传递参数 `x`;
 - 过程调用 `call p, n`: 调用 `p` 过程, 实参个数为 `n`;
 - `y = call p, n`: 函数调用——调用 `p` 过程, 返回值为 `y`;



[x;
过程，实参个数为 n;
—调用 p 过程，返回值为 y;

何为?

- 例

过程调用 `heapsort(m, n)` 对应的三地址码为？



- ① 知识点回顾
- ② 中间代码生成阶段
- ③ 中间代码的主要形式
- ④ 三地址码的表示与存储

三地址码的表示方法之四元式

- 三地址码说明了各类指令的组成部分，但未描述指令在**某个数据结构中的表示**方法。它可以用**四元式**、**三元式**和**间接三元式**来描述。
- 四元式是一个带有 4 个字段的记录结构：op, arg1, arg2, result, 其中 arg1, arg2 和 result 通常指向符号表项目的入口。
- 特别地，有一些项目无法使用四元组表示，即四元式特例：
 - $x = op\ y$ 和 $x = y$ ：不使用 arg2
 - param 运算：不使用 arg2 和 result
 - 转移指令：将目标标号 (label) 存入 result

例 $(a=b*(-c)+b*(-c))$ 的四元式

四元式表示：

AST 的三地址码：

```
t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
```

序号	op	arg1	arg2	result
(0)	uminus	c	N/A	t1
(1)	*	b	t1	t2
(2)	uminus	c	N/A	t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5	N/A	a

四元式更多例子

三地址码	四元式				
x[i]=y	[i]=	x	i	t1	
	=	y	N/A	t1	
y=x[i]	=[]	x	i	t1	
	=	t1	N/A	y	
goto L	go	N/A	N/A	L	
if a<b goto L		J<	a	b	L

三地址码之三元式

- 三元式是 3 个字段的记录结构：op, arg1, arg2;
- 三元式可以避免引入临时变量（如 t1, t2, t3）;
- 使用运算 $x \text{ op } y$ 的位置来引用它的运算结果;
- 用带有括号的数字表示指向三元式结构的指针;

例 $(a=b*(-c)+b*(-c))$ 的三元式

四元式表示:

三元式表示法:

AST 的三地址码:

```
t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
```

序号	op	arg1	arg2	result
(0)	uminus	c	N/A	t1
(1)	*	b	t1	t2
(2)	uminus	c	N/A	t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5	N/A	a

序号	op	arg1	arg2
(0)	uminus	c	—
(1)	*	b	(0)
(2)	uminus	c	—
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

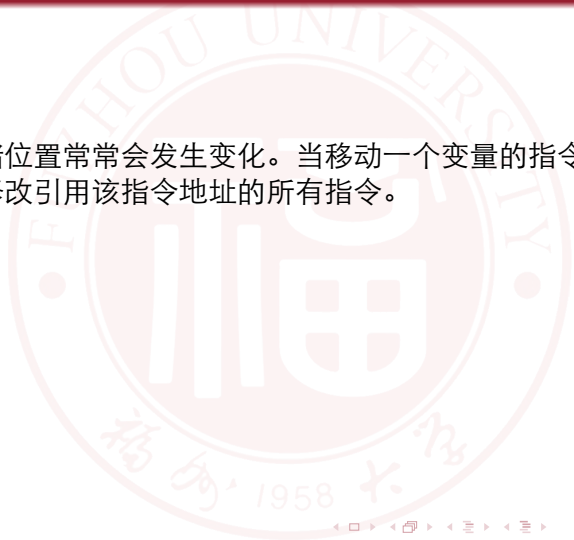
三元式更多例子

三地址码	四元式					三元式			
$x[i]=y$		$[] =$	x	i	$t1$	(1)	$[] =$	x	i
		$=$	y	N/A	$t1$	(2)	$=$	(1)	y
$y=x[i]$		$=[]$	x	i	$t1$	(1)	$=[]$	x	i
		$=$	$t1$	N/A	y	(2)	$=$	y	(1)

三地址码之间接三元式

三元式表示法的问题：

- 在优化编译器中，指令实际存储位置常常会发生变化。当移动一个变量的指令时，使用三元式表示法便需要修改引用该指令地址的所有指令。



三地址码之间接三元式

三元式表示法的问题：

- 在优化编译器中，指令实际存储位置常常会发生变化。当移动一个变量的指令时，使用三元式表示法便需要修改引用该指令地址的所有指令。

解决方案——间接三元式：

- 间接三元式方案包括三元式表 + 间接码表；
- 间接码表（指针数组）：一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置，优化编译器可以通过对该表的重新排序来移动指令的位置。
- 优点：方便优化，节省空间

例 $(a=b*(-c)+b*(-c))$ 的间接三元式

	statement
(14)	(0)
(15)	(1)
(16)	(2)
(17)	(3)

	op	arg1	arg2
(0)	uminus	c	—
(1)	*	b	(0)
(2)	+	(1)	(1)
(3)	=	a	(2)

综合示例

例

构造下述语句的三地址码、四元式、三元式、间接三元式：

$X = (A + B) * C;$

$Y = D \uparrow (A + B);$